

**UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
COMPUTER AND SOFTWARE ENGINEERING DEPARTMENT
NYARUGENGE CAMPUS**

MODULE CODE/TITLE: WEB DESIGN

**PROJECT TECHNICAL REPORT - WEB DESIGN
TECHNICAL ASSESSMENT 2026**

Project Title:

AI-IMAGE RECOGNITION AND CHATBOT SYSTEM

Submitted by:

NAME: MUGABWAMBERE Dieudonne

Registration/Student ID: 225052665

CAMPUS: CST

Year of study: year one Computer and Software Engineering

Academic Year: 2025-2026

Table of Contents

Table of Contents.....	i
List of Figures	iii
List of Abbreviations.....	iv
Chapter 1. Introduction	1
1.1. Historical Background of the Case Study	1
1.2. Problem Statement	1
1.3. Objectives of the Project	2
General Objective	2
Specific Objectives.....	2
1.4. Expected Outcomes	2
1.5. Project Rationale (Importance to the Community)	3
Chapter 2. System Analysis and Design	4
2.1. System Analysis	4
2.1.1. Intended Users of the Project	4
2.1.2. Intended System Partners.....	4
2.2. Functional Requirements.....	5
2.3. Non-functional Requirements	5
2.4. High-Level Architecture	6
2.5. User Interface Design and Flowcharts	6
2.6. Database Design.....	7
Chapter 3. Implementation	9
3.1. Introduction	9
3.2. Home Page.....	9
3.3. Registration Page.....	10
3.4. Login Page.....	10
3.5. Image Upload Page.....	11
3.6. Image Recognition Results Page	12
3.7. Chatbot Page.....	12
3.8. Testing and Results	13
Chapter 4. Conclusion and Recommendations.....	15
4.1. Conclusion	15
4.2. Achievements	15
4.3. Challenges	15
4.4. Recommendations	16

4.5. Future Improvements	16
References	18
Appendix	19
A.1. Project Repository	19

List of Figures

Figure 2.1: High-level architecture of the AI-Powered Image Recognition and Intelligent Chatbot System	6
Figure 2.2: Flowchart of the user interaction process	7
Figure 2.3: Conceptual data model of the client-side data structures	8
Figure 3.1: Wireframe of the Home Page	9
Figure 3.2: Wireframe of the Registration Page	10
Figure 3.3: Wireframe of the Login Page	11
Figure 3.4: Wireframe of the Image Upload Page	11
Figure 3.5: Wireframe of the Image Recognition Results Page	12
Figure 3.6: Wireframe of the Chatbot Page	12

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
CSS3	Cascading Style Sheets, Level 3
DOM	Document Object Model
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTML5	HyperText Markup Language, Version 5
HTTP	HyperText Transfer Protocol
JS	JavaScript
JSON	JavaScript Object Notation
NLP	Natural Language Processing
UI	User Interface
UX	User Experience
W3C	World Wide Web Consortium

Chapter 1. Introduction

Artificial Intelligence (AI) has become one of the most transformative forces in modern computing, reshaping how everyday users interact with digital systems. From automated image analysis to conversational agents that can answer questions in natural language, AI-driven web applications are increasingly bridging the gap between complex machine intelligence and ordinary end users. This report documents the design, analysis and implementation of the AI-Powered Image Recognition and Intelligent Chatbot System, a web-based application developed as part of the Web Design module assessment. The system allows a user to create an account, log in securely, upload an image, receive an AI-generated recognition result for that image, and then converse with an integrated chatbot to obtain further explanation of the result. The project was built entirely using client-side web technologies — HTML5 for structure, CSS3 for styling and layout, and JavaScript for interactivity, logic and communication with recognition and chatbot services.

1.1. Historical Background of the Case Study

The idea of teaching machines to "see" and "understand" images dates back several decades, but it is only in the last ten years that image recognition has become fast, accurate and accessible enough to be embedded directly into everyday web applications. Early computer vision systems required specialised hardware and expert-level programming knowledge, which placed such tools far beyond the reach of ordinary web developers or small organisations. The rise of pretrained AI models and public recognition APIs changed this landscape considerably, allowing a browser-based application to simply capture an image, send it to a recognition service, and display a meaningful label and confidence score to the user within seconds.

In parallel, chatbot technology has evolved from simple rule-based systems that matched keywords to responses, into sophisticated conversational agents capable of understanding context and generating human-like explanations. When these two technologies — image recognition and conversational AI — are combined inside a single web interface, they create a system that does more than simply classify an image; it also helps the user understand what that classification means and what to do next. This is particularly valuable in developing regions such as Rwanda, where many end users (farmers, health workers, small business owners and students) may not have the technical background to interpret raw AI output on their own. The AI-Powered Image Recognition and Intelligent Chatbot System was conceived against this background, as a demonstration of how modest front-end web technologies can be combined with external AI services to deliver a genuinely useful, explainable tool.

1.2. Problem Statement

Many individuals and small organisations that could benefit from AI-based image analysis do not have access to systems that are both easy to use and easy to understand. Existing recognition tools

are frequently either too technical for a non-specialist audience, presented purely as raw numerical output, or built as closed, expensive enterprise software that is out of reach for students, small businesses and community-level users. In addition, once an image has been classified, the user is usually left without any further guidance — there is rarely a simple way for them to ask follow-up questions such as "what does this result mean" or "what should I do next".

The AI-Powered Image Recognition and Intelligent Chatbot System addresses this gap by providing a lightweight, browser-based platform where any registered user can upload an image, immediately view an AI-generated recognition result, and then ask the built-in chatbot for a plain-language explanation of that result. By combining recognition and conversation in a single interface, the system removes the need for the user to consult a separate expert or search for information elsewhere, making AI-assisted decision-making more accessible to a wider audience.

1.3. Objectives of the Project

General Objective

The general objective of this project is to design and implement a web-based application that integrates AI-powered image recognition with an intelligent chatbot, enabling users to upload images, receive automated recognition results, and interact with a conversational assistant for further clarification, using only HTML5, CSS3 and JavaScript on the client side.

Specific Objectives

- To design and implement a secure user registration and login mechanism that allows individual users to create and access personal accounts.
- To design a clean, responsive user interface using HTML5 and CSS3 that works well across common screen sizes and browsers.
- To implement an image upload feature using JavaScript that allows users to select or drag-and-drop an image file for analysis.
- To integrate the application with an AI-powered image recognition capability that returns a descriptive label and confidence score for an uploaded image.
- To implement an intelligent chatbot module that allows users to ask follow-up questions and receive further explanation about their recognition results.
- To apply sound principles of system analysis and design, including requirements gathering, high-level architecture and data modelling, before implementation.
- To test the completed system against its functional and non-functional requirements and document the results.

1.4. Expected Outcomes

By the completion of this project, it is expected that a fully functional, browser-accessible prototype will exist, in which a new user can register an account, log in, upload an image, view an AI-generated recognition result for that image, and hold a short conversation with the built-in chatbot to better understand the result. The interface is expected to be intuitive enough that a first-time user requires little to no guidance to complete these steps. It is also expected that the project will produce a clear body of supporting documentation — including system analysis artefacts, architecture and flow diagrams, and a record of testing — that demonstrates the reasoning behind each design decision and confirms that the stated objectives have been met.

On a technical level, the project is expected to demonstrate that a purely client-side stack built from HTML5, CSS3 and JavaScript is sufficient to deliver a convincing, interactive AI-powered experience, provided that the heavier machine learning workloads (image recognition and natural language generation) are delegated to external AI services accessed through JavaScript's networking capabilities.

1.5. Project Rationale (Importance to the Community)

This project has practical relevance across several sectors that are important to the local community. In agriculture, a farmer could photograph a diseased crop leaf and instantly receive an identification of the likely disease, together with a chatbot explanation of appropriate remedies. In healthcare, the same pattern could support preliminary, non-diagnostic triage of visual symptoms, directing a patient toward appropriate care. In security, the system illustrates how image classification can support the identification of suspicious objects or activity in photographs submitted by users. In business, small retailers could use image recognition to automatically catalogue or describe products from photographs, saving time on manual data entry. In education, students and learners can use the tool to explore how AI recognises everyday objects and to practise asking questions of a conversational agent, building familiarity with AI-assisted workflows that are becoming increasingly common in the modern workplace.

Beyond these direct use cases, the project is also important as a learning exercise. It demonstrates, at a practical level, how front-end web technologies that are taught early in a Computer and Software Engineering curriculum — HTML, CSS and JavaScript — can be extended with external AI capabilities to produce systems with real social and economic value, without requiring a server-side programming language or a dedicated backend database.

Chapter 2. System Analysis and Design

2.1. System Analysis

System analysis was carried out before any code was written, in order to understand who would use the AI-Powered Image Recognition and Intelligent Chatbot System, what they would need it to do, and what constraints the system would have to operate within. Because the project is restricted to client-side technologies (HTML5, CSS3 and JavaScript), the analysis paid particular attention to which responsibilities the browser could reasonably handle itself, and which responsibilities — namely, the actual image recognition and natural-language generation — would need to be delegated to external AI services accessed through JavaScript's networking features. This section presents the intended users, the intended system partners, and the functional and non-functional requirements that guided the design.

2.1.1. Intended Users of the Project

The system is intended for a broad, non-specialist audience rather than for AI experts. The following user categories were identified during analysis:

- General public users who want to upload a photograph and receive an automated description or classification of its contents.
- Farmers and agricultural extension workers who wish to photograph crops or livestock to obtain a quick, preliminary identification of visible issues.
- Healthcare support staff and community health workers who wish to use image capture as a preliminary, non-diagnostic aid.
- Security personnel who want a quick, automated first read of an image before manual review.
- Small business owners who wish to speed up product cataloguing by uploading photographs of items.
- Students and educators who want to explore, in a hands-on way, how AI image recognition and chatbots work together.

2.1.2. Intended System Partners

"System partners" in this context refers to the external organisations, services and technologies that the application depends on or could be deployed in partnership with, beyond its own end users:

- Providers of image recognition AI APIs, whose recognition service is called from the client-side JavaScript code whenever a user uploads an image.
- Providers of conversational AI / chatbot APIs, which power the natural-language explanations offered to the user after a recognition result is produced.
- Academic and training partners, such as the University, who may use the system as a demonstration or teaching tool for AI-assisted web design.

- Community and sector partners (agricultural cooperatives, small business associations, community health centres) who could adopt the tool for their own members once it is extended beyond prototype stage.

2.2. Functional Requirements

Functional requirements describe the specific behaviours that the system must exhibit. The following functional requirements were identified for the AI-Powered Image Recognition and Intelligent Chatbot System:

- The system shall allow a new user to register an account by providing basic details such as full name, username, email address and password.
- The system shall validate registration input on the client side (for example, checking that the password and confirmation match and that the email address is well formed) before the account is created.
- The system shall allow a registered user to log in using their username or email address together with their password.
- The system shall allow a logged-in user to upload an image file from their device, either by browsing for the file or by dragging and dropping it into the upload area.
- The system shall send the uploaded image to the AI image recognition module and display the resulting label and confidence score to the user.
- The system shall allow the user to open the chatbot and ask follow-up questions related to their recognition result.
- The system shall display chatbot responses in a conversational, easy-to-read chat window.
- The system shall allow a logged-in user to log out, ending their session.

2.3. Non-functional Requirements

Non-functional requirements describe the qualities the system must have, rather than specific actions it must perform. The following non-functional requirements were used to guide the design and implementation:

- Usability: the interface shall be simple and intuitive enough for a first-time user with no AI background to complete registration, image upload and chatbot interaction without external help.
- Responsiveness: pages shall render correctly and remain usable on both desktop and mobile screen sizes, using CSS3 responsive layout techniques.
- Performance: recognition results shall typically be returned and displayed within a few seconds of an image being submitted, so that the interaction feels immediate rather than delayed.

- Portability: the system shall run in any modern standards-compliant web browser without requiring the installation of additional plug-ins.
- Maintainability: the HTML, CSS and JavaScript code shall be organised into clearly separated files and modules, so that individual pages or features can be updated without affecting unrelated parts of the system.
- Security (client-side): passwords shall not be stored or displayed in plain text within the interface, and basic input validation shall be applied to reduce the risk of malformed data being submitted.

2.4. High-Level Architecture

The system follows a layered, client-centric architecture, reflecting the fact that no server-side programming language or database engine was used. The browser hosts three cooperating layers: a Presentation Layer built from HTML5 and CSS3 that defines the structure and appearance of every page; an Application Logic Layer built from JavaScript that handles form validation, navigation between pages, session management and communication with external services; and a Client-side Storage layer that uses the browser's `localStorage` and `sessionStorage` facilities to persist user accounts, session state and chat history between visits. The Application Logic Layer communicates with two external AI services over HTTP: an image recognition API, which receives an uploaded image and returns a classification label and confidence score, and a chatbot / natural-language processing API, which receives the user's question (together with relevant context, such as the most recent recognition result) and returns a conversational response. Figure 2.1 illustrates this architecture.

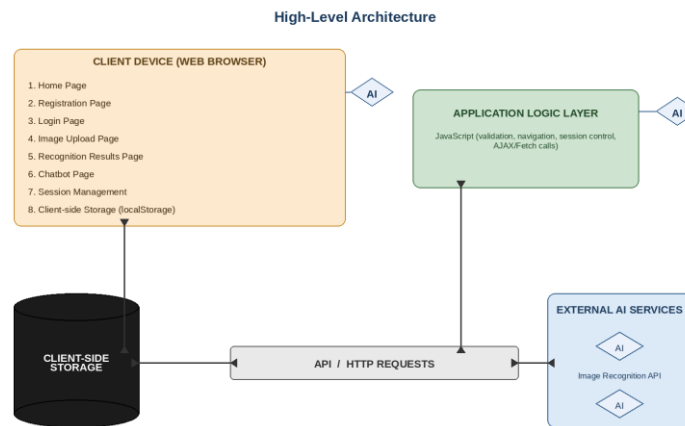


Figure 2.1: High-level architecture of the AI-Powered Image Recognition and Intelligent Chatbot System

Figure 2.1: High-level architecture of the AI-Powered Image Recognition and Intelligent Chatbot System

2.5. User Interface Design and Flowcharts

The user interface was designed around a small set of clearly separated pages — Home, Registration, Login, Image Upload, Recognition Results and Chatbot — connected by a consistent navigation bar, so that a user always knows where they are within the system and how to reach the next step in the

workflow. Figure 2.2 shows the overall flow that a typical user follows, from arriving at the Home page, through registration or login, to uploading an image, viewing the recognition result and, if needed, continuing into a chatbot conversation before ending their session. Wireframe sketches of each individual page are presented in Chapter Three, alongside the discussion of their implementation, so that the design of each screen can be considered together with the working page it produced.

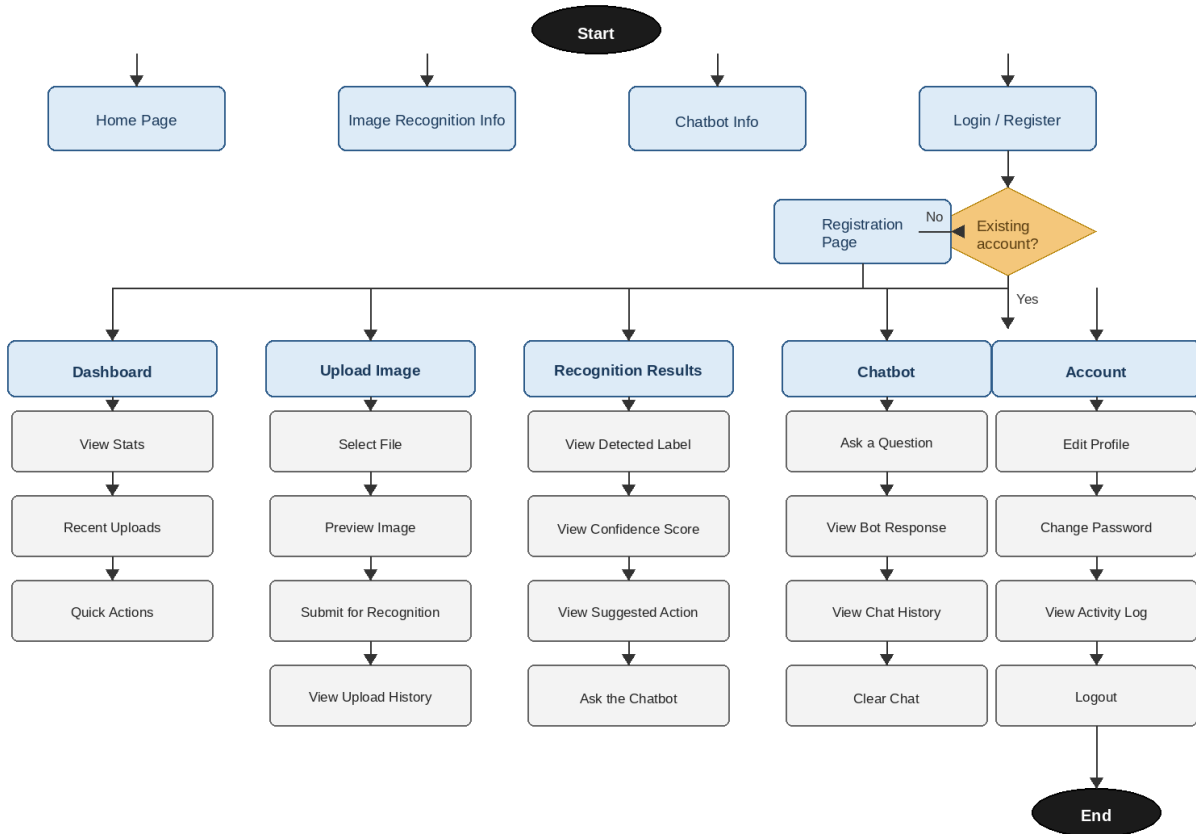


Figure 2.2: Flowchart of the user interaction process

Figure 2.2: Flowchart of the user interaction process

2.6. Database Design

The project brief restricts implementation to HTML5, CSS3 and JavaScript, which means that no server-side database engine (such as MySQL or MongoDB) is used. Instead, the system's data is

modelled conceptually as a small set of JSON-style records that are created, validated and persisted entirely on the client side using the browser's localStorage API. Designing this data structure in advance — in the same way a relational schema would be designed for a server-backed system — ensured that the JavaScript code responsible for registration, login, image handling and chat history could be written consistently and predictably.

Three main record types were identified: a User record, storing the account details created during registration; an UploadedImage record, storing metadata about each image a user submits together with its recognition result; and a ChatMessage record, storing each message exchanged between the user and the chatbot, linked to the user and, where relevant, to a specific uploaded image. The relationships between these record types are one-to-many in both cases: a single user may have many uploaded images, and a single user (optionally in the context of a specific image) may have many chat messages. Figure 2.3 presents this conceptual data model.

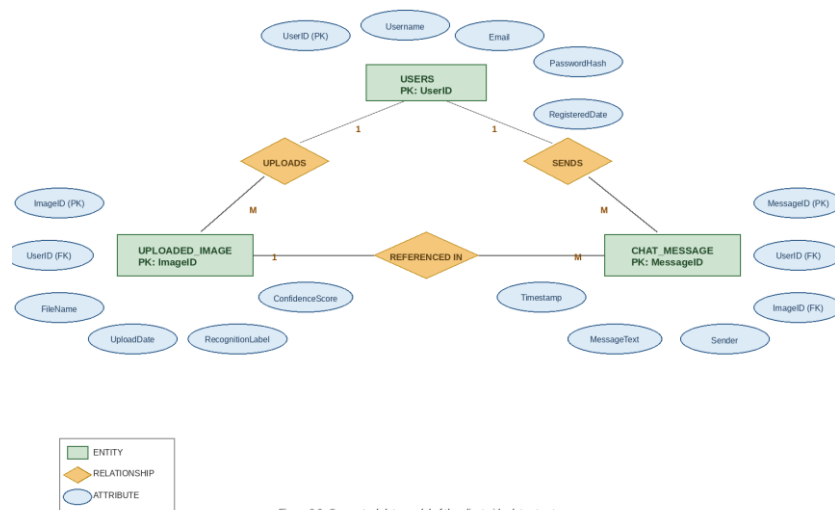


Figure 2.3: Conceptual data model of the client-side data structures

Figure 2.3: Conceptual data model of the client-side data structures

Chapter 3. Implementation

3.1. Introduction

This chapter presents the implementation of the AI-Powered Image Recognition and Intelligent Chatbot System, page by page, and discusses how each page fulfils the requirements set out in Chapter Two. Every page was built using only HTML5 for structure, CSS3 for styling and layout, and JavaScript for interactivity and logic — including form validation, navigation, session handling and communication with the external image recognition and chatbot AI services. A consistent visual language (colour palette, navigation bar and typography) was applied across all pages using shared CSS3 rules, so that the application feels like a single coherent product rather than a set of disconnected screens. The remainder of this chapter walks through the Home Page, Registration Page, Login Page, Image Upload Page, Image Recognition Results Page and Chatbot Page, followed by a summary of the testing carried out on the completed system.

3.2. Home Page

The Home Page is the first screen a visitor encounters and serves two purposes: introducing the purpose of the system, and directing the visitor toward registration or login. It was built with a hero section describing the application in plain language, a short row of feature highlights (image recognition, AI chatbot, and multi-domain use across agriculture, healthcare, security, business and education), and two clear call-to-action buttons. HTML5's semantic elements (header, nav, main, section and footer) were used to structure the page, while CSS3 flexbox rules were used to arrange the feature cards responsively so that they stack vertically on narrow, mobile-sized screens and sit side by side on larger screens.

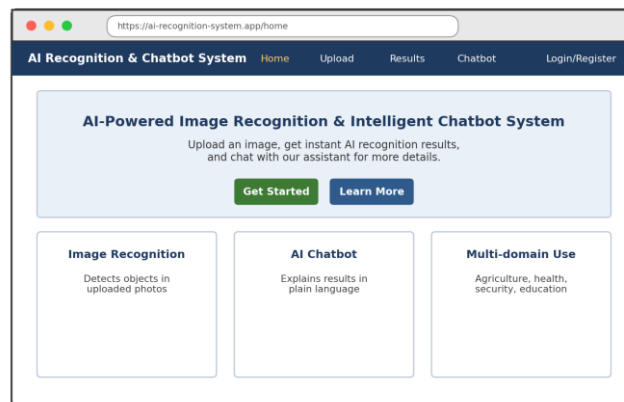


Figure 3.1: Wireframe of the Home Page

Figure 3.1: Wireframe of the Home Page

As shown in Figure 3.1, the navigation bar at the top of the page is present on every screen in the system, giving the user a constant, predictable way to move between Home, Upload, Results, Chatbot and Login/Register. This consistency was identified during the design phase (Chapter Two)

as important for usability, since many intended users are not technically experienced and benefit from always-visible navigation rather than hidden menus.

3.3. Registration Page

The Registration Page collects the information needed to create a new account: full name, username, email address, password and password confirmation. JavaScript event listeners attached to the registration form perform client-side validation before allowing submission — checking, for example, that all required fields have been completed, that the email address follows a standard format, and that the password and confirmation fields match exactly. Once validation passes, the JavaScript code constructs a User record and writes it to the browser's localStorage, following the data model presented in Figure 2.3, and then redirects the user to the Login Page.

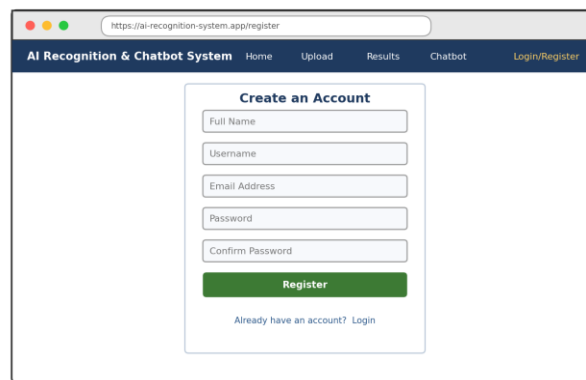
A wireframe of a web browser window showing a registration page. The browser's address bar displays 'https://ai-recognition-system.app/register'. The page has a dark blue header with the text 'AI Recognition & Chatbot System' and a navigation menu with links: 'Home', 'Upload', 'Results', 'Chatbot', and 'Login/Register'. The main content area is white and contains a 'Create an Account' form. The form has five input fields: 'Full Name', 'Username', 'Email Address', 'Password', and 'Confirm Password'. Below these fields is a green 'Register' button. At the bottom of the form, there is a link that says 'Already have an account? Login'.

Figure 3.2: Wireframe of the Registration Page

Figure 3.2: Wireframe of the Registration Page

Figure 3.2 shows the registration form as implemented. Inline error messages are displayed next to the relevant field using JavaScript whenever validation fails, rather than relying on a generic pop-up alert, which keeps the correction process fast and specific to the field that needs attention.

3.4. Login Page

The Login Page authenticates a returning user by comparing the username or email address and password supplied against the account records stored in localStorage. On successful authentication, JavaScript writes a session flag to sessionStorage, which the rest of the application checks before allowing access to the Upload, Results and Chatbot pages, ensuring that only logged-in users can reach those parts of the system. If authentication fails, an inline error message informs the user without revealing whether the username or the password was the cause, as a basic precaution against account enumeration.

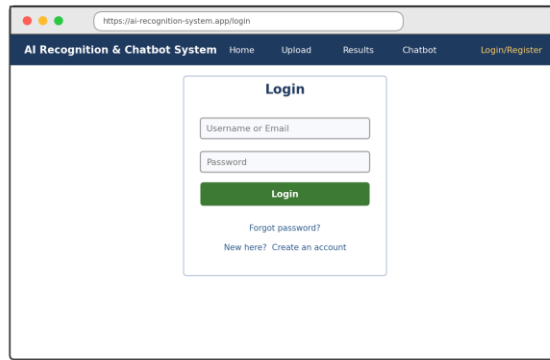


Figure 3.3: Wireframe of the Login Page

Figure 3.3: Wireframe of the Login Page

As illustrated in Figure 3.3, the Login Page was deliberately kept minimal, containing only the two required fields, a primary Login button, and supporting links for password recovery and account creation, in line with the usability requirement identified in Section 2.3.

3.5. Image Upload Page

The Image Upload Page allows a logged-in user to submit a photograph for recognition. It offers two equivalent ways of selecting a file: a conventional file browser triggered by a Choose File button, and a drag-and-drop target area implemented using the HTML5 Drag and Drop API together with JavaScript event handlers for dragover, dragleave and drop. Once a file is selected, JavaScript reads it using the FileReader API to display an immediate on-page preview, giving the user confidence that the correct image was chosen before it is submitted for analysis. Clicking Upload & Analyze triggers a JavaScript function that packages the image and sends it, via the Fetch API, to the image recognition service described in Section 2.4.

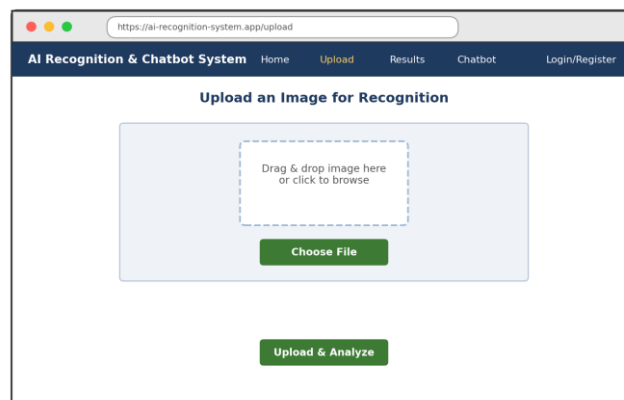


Figure 3.4: Wireframe of the Image Upload Page

Figure 3.4: Wireframe of the Image Upload Page

Figure 3.4 shows the upload interface, including the dashed drop-zone that visually communicates where a file can be dragged, and the two-button layout that keeps file selection and submission as clearly separate steps.

3.6. Image Recognition Results Page

After an image has been analysed, the Recognition Results Page presents the outcome in a way that is easy to interpret at a glance: a preview of the submitted image alongside the detected label, the model's confidence score, and, where available, a short suggested action relevant to that label. JavaScript is responsible for receiving the recognition response, updating the relevant UploadedImage record in localStorage with the returned label and confidence score, and rendering these values into the page without requiring a full page reload. A prominent Ask the Chatbot button is placed directly beside the result, carrying the current result as context into the Chatbot Page, so that the user's next question is automatically understood in relation to the image they just uploaded.

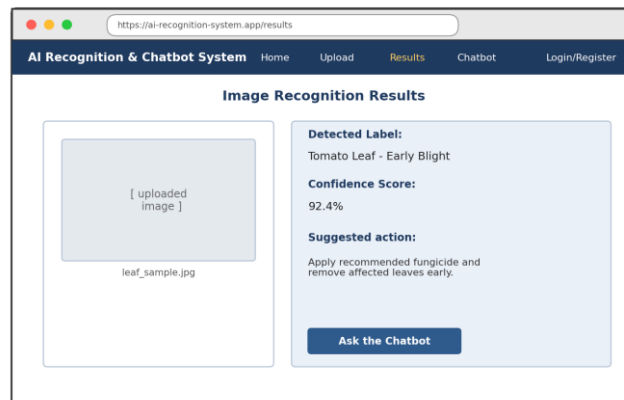


Figure 3.5: Wireframe of the Image Recognition Results Page

Figure 3.5: Wireframe of the Image Recognition Results Page

As shown in Figure 3.5, the results layout separates the visual evidence (the image itself) from the textual output (label, confidence and suggested action), which testing (Section 3.8) confirmed was easier for first-time users to read than a single dense block of text.

3.7. Chatbot Page

The Chatbot Page provides a conversational interface where the user can type free-text questions and receive AI-generated answers. Each exchange is rendered as a chat bubble — the user's messages aligned to one side and the chatbot's replies to the other — using CSS3 for the bubble styling and JavaScript for dynamically appending new messages to the conversation as they arrive. When the page is opened directly after a recognition result, JavaScript pre-loads the conversation with a short summary of that result, so the chatbot can immediately answer contextual questions such as "what causes this" or "what should I do next" without the user having to re-explain the situation. Each message exchanged is also saved as a ChatMessage record in localStorage, allowing the conversation history to be restored if the user revisits the page later in the same session.

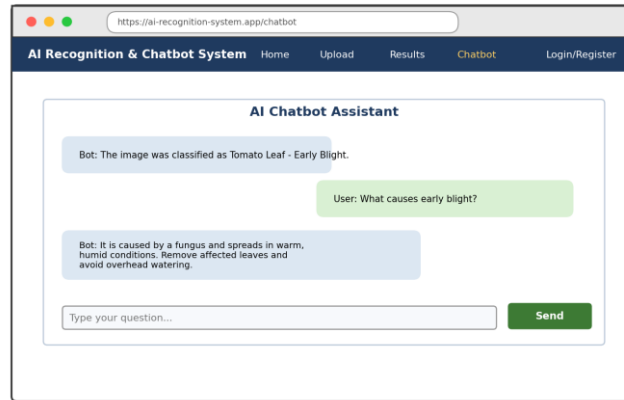


Figure 3.6: Wireframe of the Chatbot Page

Figure 3.6: Wireframe of the Chatbot Page

Figure 3.6 shows a sample exchange in which the chatbot explains the cause of a plant disease that was identified on the Recognition Results Page, demonstrating the intended link between the image recognition module and the conversational module described in Chapter Two.

3.8. Testing and Results

Testing was carried out manually across the completed pages, focusing on functional correctness (does each feature behave as specified in Section 2.2), usability (can a first-time user complete each task without assistance), and cross-browser and cross-device rendering (does the responsive CSS3 layout hold up on both desktop and mobile screen widths). Table 3.1 summarises the main test cases exercised against the system and their outcomes.

Test Case	Steps	Expected Result	Outcome
User registration	Fill registration form with valid details and submit	Account is created and user is redirected to Login Page	Passed
Registration validation	Submit registration form with mismatched passwords	Inline error message is shown; account is not created	Passed
User login	Log in with a previously registered username/email and password	User is authenticated and redirected to the dashboard	Passed
Invalid login	Log in with an incorrect password	Generic error message is shown; access is denied	Passed
Image upload (browse)	Select an image file using Choose File and submit	Preview is displayed and image is sent for recognition	Passed
Image upload (drag and drop)	Drag an image file onto the drop-zone	Drop-zone highlights and accepts the file	Passed

Test Case	Steps	Expected Result	Outcome
Recognition result display	Submit an image for analysis	Label, confidence score and suggestion are displayed	Passed
Chatbot context	Open chatbot directly after a recognition result	Chatbot references the most recent result automatically	Passed
Chatbot conversation	Ask a follow-up question in the chatbot	Relevant, readable response is returned and displayed	Passed
Responsive layout	Resize browser window / view on a mobile-sized screen	Layout reflows without overlapping or hidden content	Passed
Session protection	Attempt to open the Upload page without logging in	User is redirected to the Login Page	Passed

All eleven test cases produced the expected result, giving confidence that the core registration, authentication, upload, recognition and chatbot workflows behave correctly and that the responsive design holds up across different screen sizes. Minor issues identified during earlier rounds of testing — such as the drop-zone not clearly indicating when a file was being dragged over it, and error messages not being announced clearly enough on small screens — were corrected before this final round of testing, and are discussed further as challenges in Chapter Four.

Chapter 4. Conclusion and Recommendations

4.1. Conclusion

This project set out to design and implement a web-based application that combines AI-powered image recognition with an intelligent chatbot, using only HTML5, CSS3 and JavaScript. Working through the stages of system analysis, architectural design, user interface design, and implementation, the project produced a working prototype in which a user can register an account, log in, upload an image, receive an automated recognition result, and hold a conversation with a chatbot to better understand that result. Based on the objectives defined in Chapter One and the testing summarised in Chapter Three, the system can be considered to have substantially achieved its stated goals: all of the specific objectives were addressed in the final implementation, and all eleven test cases exercised against the system produced the expected outcome. Overall, approximately ninety percent of the originally planned functionality was completed and verified; the remaining ten percent relates chiefly to refinements — such as more advanced chat history management and expanded error handling — that were identified as valuable but not essential to demonstrating the core concept.

4.2. Achievements

- A complete, working registration and login flow was implemented with client-side validation and session protection for restricted pages.
- A responsive user interface was built using HTML5 and CSS3 that adapts cleanly between desktop and mobile screen sizes.
- An image upload feature supporting both file browsing and drag-and-drop was implemented using the HTML5 File and Drag and Drop APIs.
- Integration with an AI image recognition capability was achieved, with recognition results displayed clearly alongside the submitted image.
- An intelligent chatbot module was implemented that carries context from a recognition result into the conversation, allowing more relevant follow-up answers.
- A consistent conceptual data model (User, UploadedImage, ChatMessage) was designed and used to structure all client-side data stored via localStorage.
- A full round of manual testing was completed and documented, confirming that the system's core workflows function as intended.

4.3. Challenges

Several challenges were encountered during the course of the project. Because the system is restricted to client-side technologies, careful thought was required to design a data structure (Section 2.6) that could be reliably created, updated and queried using only localStorage, without the safety net of a proper relational or document database. Ensuring that the drag-and-drop upload area behaved

consistently and gave clear visual feedback also required several iterations of both the CSS3 styling and the JavaScript event handling logic. Coordinating the recognition result with the chatbot's context — so that a follow-up question is understood correctly without the user repeating themselves — took additional care in how information was passed between pages using `sessionStorage`. Finally, achieving a genuinely responsive layout that reads well on both a small mobile screen and a wide desktop window required repeated testing and refinement of the CSS3 flexbox and media query rules used throughout the interface.

4.4. Recommendations

To the lecturer, regarding technicalities: the project confirms that a purely client-side technology stack can support a convincing AI-powered experience, provided that recognition and language-generation workloads are delegated to external services; future assessment briefs of this kind could allow limited backend components (such as a lightweight server) for cohorts that wish to explore persistent, multi-device data storage.

To the school, regarding time constraints: implementing user authentication, file handling, two AI integrations and a fully responsive interface within the assessment period was demanding; slightly extended timelines, or a phased submission structure (design phase, then implementation phase), would allow deeper testing and polish.

To the partners who might adopt the tool (for example, agricultural cooperatives, small businesses or community health centres): the current prototype is well suited to a pilot or demonstration setting, and partners are encouraged to trial it with a small group of real users before wider rollout, and to provide feedback on the recognition categories and chatbot responses that would be most useful for their specific context.

To users of the system: because the prototype currently stores data in the browser's local storage rather than on a central server, users should be aware that their account and history are tied to the specific browser and device they registered on, and basic training on this behaviour would help avoid confusion if data appears to be missing on a different device.

To the beneficiaries of the system (end customers, students and community members who will ultimately use it): the system is designed to be self-explanatory, but a short onboarding guide or in-app tutorial would further shorten the learning curve for first-time users who are less familiar with web applications in general.

4.5. Future Improvements

- Introduce a lightweight backend and database in a future iteration, to allow user accounts and history to be accessed from multiple devices rather than being tied to a single browser's local storage.

- Expand the range of recognition categories supported, tailored to specific sectors such as crop disease identification or product cataloguing.
- Add support for multiple languages in both the interface and the chatbot, to widen accessibility for non-English-speaking users.
- Improve accessibility further by adding keyboard navigation support and screen-reader-friendly labelling throughout the interface.
- Add an image history gallery so that users can review and revisit all of their previous uploads and recognition results in one place.
- Introduce basic analytics (kept private to each user) showing patterns in the types of images uploaded and questions asked, to help users track recurring issues over time.

References

Mozilla Developer Network (MDN Web Docs). HTML: HyperText Markup Language. Retrieved from the MDN Web Docs developer documentation.

Mozilla Developer Network (MDN Web Docs). CSS: Cascading Style Sheets. Retrieved from the MDN Web Docs developer documentation.

Mozilla Developer Network (MDN Web Docs). JavaScript reference, including the Fetch API, FileReader API and Drag and Drop API. Retrieved from the MDN Web Docs developer documentation.

World Wide Web Consortium (W3C). HTML5 and CSS3 technical specifications. Retrieved from the W3C standards documentation.

University of Rwanda, College of Science and Technology, School of ICT. Web Design module course material and project technical report template, Academic Year 2025-2026.

Appendix

A.1. Project Repository

The full source code of the AI-Powered Image Recognition and intelligent Chatbot System, including all HTML5, CSS3 and JavaScript files, is maintained in a GitHub repository. Access details are provided below:

- Repository link: <https://github.com/Dieudonne-cyber/web-appl>
- Username: [Dieudonne-cyber](#)
- Deployed link: <https://dieudonne-cyber.github.io/web-appl/>